

TASK: EXCEPTION HANDLING & VALIDATION IN REST API.

Aim

To implement global exception handling and apply input validation in a RESTful application to ensure reliable data processing and meaningful error responses using Spring Boot.

Procedure

1. Create a Spring Boot Project

Use Spring Initializr

Add dependencies:

- Spring Web
- Spring Boot Validation (Jakarta Validation)
- Lombok (optional)

2. Design a Model Class

Create an entity/DTO class.

Apply validation annotations like:

- @NotNull
- @Size
- @Email

3. Create a REST Controller

Accept input using @RequestBody

Use @Valid to trigger validation

4. Handle Exceptions Globally

Create a class annotated with @RestControllerAdvice

Handle:

Validation errors (MethodArgumentNotValidException)

Custom exceptions

Generic exceptions

5. Define Custom Exception (Optional)

Create user-defined exceptions (e.g., `ResourceNotFoundException`)

6. Return Structured Error Responses

Include:

- Timestamp
- Status code
- Error message
- Path

7. Test the API

Use Postman or curl

Send invalid inputs and observe responses

Code Implementation

1. Model Class with Validation

```
import jakarta.validation.constraints.*;
```

```
public class User {
```

```
    @NotNull(message = "ID cannot be null")
```

```
    private Long id;
```

```
    @NotBlank(message = "Name is required")
```

```
    @Size(min = 3, message = "Name must be at least 3 characters")
```

```
    private String name;
```

```
    @Email(message = "Invalid email format")
```

```
    private String email;
```

```
    @Min(value = 18, message = "Age must be at least 18")
```

```
    private int age;
```

```
    // Getters and Setters  
}
```

2. REST Controller

```
import org.springframework.web.bind.annotation.*;  
import jakarta.validation.Valid;
```

```
@RestController  
@RequestMapping("/users")  
public class UserController {  
  
    @PostMapping  
    public String createUser(@Valid @RequestBody User user) {  
        return "User created successfully";  
    }  
  
    @GetMapping("/{id}")  
    public String getUser(@PathVariable Long id) {  
        if (id <= 0) {  
            throw new ResourceNotFoundException("Invalid User ID");  
        }  
        return "User found";  
    }  
}
```

3. Custom Exception

```
public class ResourceNotFoundException extends RuntimeException {  
    public ResourceNotFoundException(String message) {  
        super(message);  
    }  
}
```

4. Global Exception Handler

```
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
```

```
import java.util.HashMap;
import java.util.Map;
```

```
@RestControllerAdvice
```

```
public class GlobalExceptionHandler {
```

```
    // Handle validation errors
```

```
    @ExceptionHandler(MethodArgumentNotValidException.class)
```

```
    @ResponseStatus(HttpStatus.BAD_REQUEST)
```

```
    public Map<String, String> handleValidationExceptions(
        MethodArgumentNotValidException ex) {
```

```
        Map<String, String> errors = new HashMap<>();
```

```
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage())
        );
```

```
        return errors;
```

```
    }
```

```
    // Handle custom exception
```

```
    @ExceptionHandler(ResourceNotFoundException.class)
```

```
    @ResponseStatus(HttpStatus.NOT_FOUND)
```

```
    public Map<String, String> handleResourceNotFound(
        ResourceNotFoundException ex) {
```

```
        Map<String, String> error = new HashMap<>();
        error.put("error", ex.getMessage());
```

```
        return error;
```

```
    }
```

```

// Handle general exceptions
@ExceptionHandler(Exception.class)
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public Map<String, String> handleGlobalException(Exception ex) {

    Map<String, String> error = new HashMap<>();
    error.put("error", "Something went wrong");

    return error;
}
}

```

5. Sample Request (Invalid Input)

POST /users

```

{
  "name": "A",
  "email": "invalid-email",
  "age": 15
}

```

6. Sample Error Response

```

{
  "name": "Name must be at least 3 characters",
  "email": "Invalid email format",
  "age": "Age must be at least 18"
}

```

Commands

Run Application

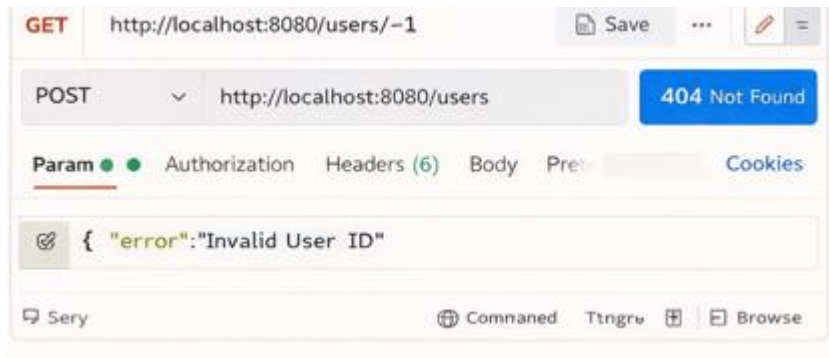
```
mvn spring-boot:run
```

Test API (curl)

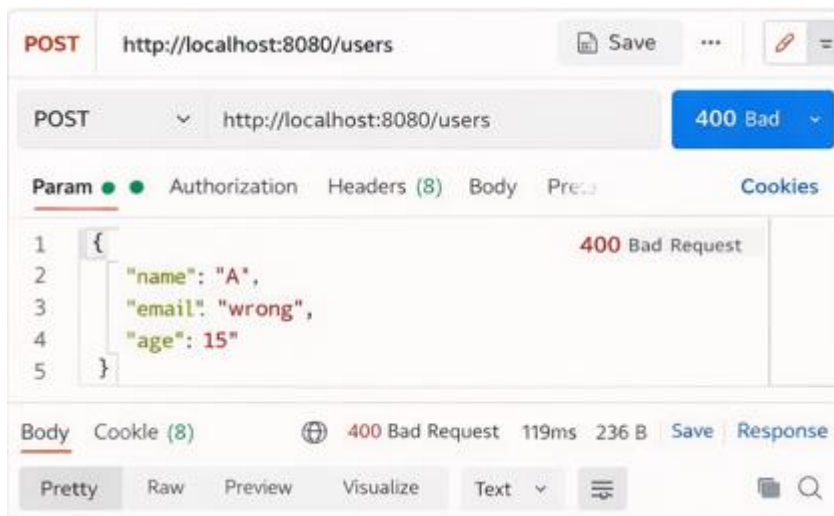
```
curl -X POST http://localhost:8080/users \
-H "Content-Type: application/json" \
-d '{"name":"A","email":"wrong","age":10}'
```

Output:

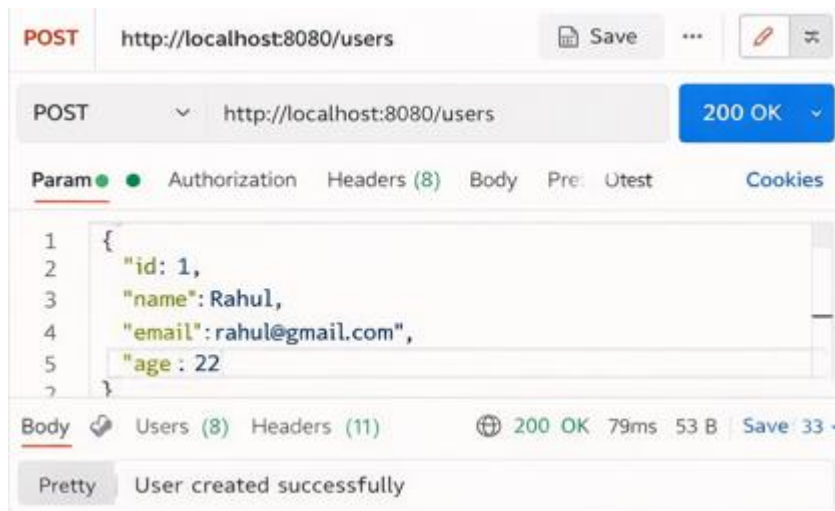
Custom exception output:



Validation error output:



Validation success output:



Result

The RESTful application successfully:

- Validates input data using annotation-based validation
- Handles exceptions globally using `@RestControllerAdvice`
- Returns structured and meaningful error responses
- Prevents invalid data from being processed

Thus, the system becomes more robust, user-friendly, and production-ready.